Mobile Money Investigations with PostgreSQL and Neo4j

Comprehensive Line-by-Line Code & Project Technical Explanation Guide

Adventist University of Central Africa (AUCA) — Master of Science in Big Data Analytics

Course: Advanced Big Data Analytics | **Lecturer:** Temitope Oguntade

Group Members:

1. Amie Marie Flora Dushimumukiza (Student ID: 101405)
2. Kwizera Rene (Student ID: 101379)
3. Grace Teta (Student ID: 101378)

1. Project Overview & Architectural Foundation

1.1 The Central Investigation Problem

In mobile money ecosystems (e.g., MTN Mobile Money, Airtel Money), financial transactions are high-volume, continuous, and complex. Investigators and regulatory bodies (such as the National Bank of Rwanda - BNR) face two fundamentally different computational challenges:

1. **Routine Relational Workloads (Tabular / Aggregate):**

- Daily network volume and fees reconciliation across 20+ million transactions.
- Account balance calculations and monthly statement summaries.
- Auditing device registration and user identity records.

2. **Forensic Network Workloads (Multi-Hop Graph Topology):**

- Tracing rapid onward payment chains (money mule networks where funds move within seconds through intermediate accounts).
- Detecting circular payment loops (funds cycling back to the origin or a co-located wallet with small percentage fee decay).
- Identifying shared device infrastructures (multiple synthetic identities or SIM cards sharing the exact same IMEI/handset).
- Finding the shortest path of fund movement between two arbitrary suspect wallets.

1.2 The Technological Showdown: Relational vs. Graph

- **PostgreSQL 18.6 (Relational Engine):** Uses table scans, B-Tree index lookups, hash joins, merge joins, and recursive Common Table Expressions (CTEs). It excels at sequential throughput and parallel data crunching (Q2 daily aggregation runs in **5.83 s** across 20 million rows), but struggles with deep recursive self-joins where combinatorial branch explosion occurs.
- **Neo4j 2026.05.0 (Graph Engine):** Uses **Index-Free Adjacency (IFA)**. In Neo4j, every node points directly to its connected edges in memory via double-linked pointer chains. Traversing a relationship does not require index lookups or table scans—it is a memory pointer dereference ($O(1)$ per hop). This makes shortest path (Q7 bound 6) complete in **18.7 ms** (compared to PostgreSQL's initial 33,668 ms, an 1,800-fold graph advantage).

2. Ingestion Pipeline & Data Model Line-by-Line

2.1 Synthetic Generator (`mobile_money_generator.py`)

The synthetic generator simulates the Rwandan mobile money economy for September 2026 (timestamps `1788235200` to `1790798399`, exactly 28 days from 1 Sep 00:00 CAT to 28 Sep 23:59 CAT).

- **Customers (228,000):** Unique legal persons with IDs from `1` to `228000`, distributed across 5 provinces (Kigali, Southern, Western, Northern, Eastern).
- **Accounts (248,000):** Wallets assigned to customers. 240,000 personal accounts, 6,000 merchant accounts, 2,000 cash-in/cash-out agent accounts.
- **Devices (228,600):** Unique mobile phone handsets / IMEIs.
- **Account Devices (285,600):** Device observation logs recording `account_id`, `device_id`, `first_seen`, and `last_seen`.
- **Transfers (20,003,300):** 20 million standard transactions plus 3,300 injected forensic patterns. Total volume: **231,482,327,870 RWF**.

2.2 Streaming Capture (`code/capture.py`) Line-by-Line

`capture.py` reads data from the generator iterator and writes clean CSV files with an explicit SHA-256 integrity audit:

```
# Lines 20-35: Setting up chunked CSV writers
def stream_to_csv():
    # 1. We instantiate generator iterators for each entity
    # 2. We use Python's csv.writer with buffer size to avoid memory exhaustion
```

- **Line-by-line mechanics:**
- `csv.writer(f, lineterminator='\n')`: Writes RFC 4180 standard CSV rows.
- `iter_transfers()`: Generates transfer records progressively in batches of 100,000 rows. At no point does the script hold 20 million rows in RAM (which would require >8 GB of memory). Memory usage stays under 150 MB throughout.
- `hashlib.sha256()`: An incremental cryptographic digest is computed on each file stream as it is written.
- `results/capture_audit.json`: Dumps exact record counts and running totals (20,003,300 transfers, 231,482,327,870 RWF).

2.3 PostgreSQL Schema & Loading (`sql/01_schema.sql` to `sql/03_constraints_indexes.sql`)

1. `01_schema.sql`:

```
CREATE TABLE customers (
    customer_id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    region TEXT NOT NULL
);
CREATE TABLE accounts (
    account_id INTEGER PRIMARY KEY,
    customer_id INTEGER NOT NULL REFERENCES customers(customer_id),
    role TEXT NOT NULL CHECK (role IN ('person', 'merchant', 'agent')),
    balance NUMERIC(14,2) NOT NULL DEFAULT 0.00
);
CREATE TABLE devices (
    device_id INTEGER PRIMARY KEY,
    brand TEXT,
    model TEXT
);
CREATE TABLE account_devices (
    account_id INTEGER NOT NULL REFERENCES accounts(account_id),
    device_id INTEGER NOT NULL REFERENCES devices(device_id),
    first_seen TIMESTAMPTZ NOT NULL,
    last_seen TIMESTAMPTZ NOT NULL,
    PRIMARY KEY (account_id, device_id, first_seen)
);
CREATE TABLE transfers (
    transfer_id BIGINT PRIMARY KEY,
    sender_id INTEGER NOT NULL REFERENCES accounts(account_id),
    receiver_id INTEGER NOT NULL REFERENCES accounts(account_id),
    amount NUMERIC(12,2) NOT NULL CHECK (amount > 0),
    occurred_at TIMESTAMPTZ NOT NULL
);
```

- **Why `CHECK (amount > 0)`:** Enforces banking business rules preventing zero or negative transfer amounts.
- **Why `TIMESTAMPTZ`:** Aligns all UTC epoch timestamps with Central Africa Time (`Africa/Kigali`, UTC+02:00).
- **Why unlogged/no-indexes during load:** Bulk loading 20 million rows into an indexed table causes B-Tree rebalancing on every row insert, slowing load times by 10x to 15x. In `02_load.sql`, tables are loaded raw via `\copy` first, and indexes are created afterwards in `03_constraints_indexes.sql`.

2. `03_constraints_indexes.sql`:

```
CREATE INDEX idx_transfers_sender ON transfers (sender_id, occurred_at);
CREATE INDEX idx_transfers_receiver ON transfers (receiver_id, occurred_at);
CREATE INDEX idx_transfers_occurred ON transfers (occurred_at);
CREATE INDEX idx_account_devices_acc ON account_devices (account_id, device_id);
CREATE INDEX idx_account_devices_dev ON account_devices (device_id, account_id);
```

- **Why composite `(sender_id, occurred_at)`:** Allows Index-Only Scans or Bitmap Index Scans for queries filtering on an account's outgoing transfers within specific date ranges.
- **Index size:** Table data: 2,684 MB; Indexes: 1,044 MB; Total database size: 3,728 MB.

2.4 Neo4j Schema & Offline Admin Import

1. Property Graph Structure:

- `(:Customer {customer_id, name, region})`
- `(:Account {account_id, role, balance})`
- `(:Device {device_id, brand, model})`
- `(Customer)-[:OWNS]->(Account)`
- `(Account)-[:USED_DEVICE {first_seen, last_seen}]->(Device)`
- `(Account)-[:TRANSFER {transfer_id, amount, occurred_at}]->(Account)`

2. High-Speed Admin Import (`neo4j-admin database import full`):

- Offline import bypasses transaction logs and database lock managers, constructing the store files directly on disk.
- Result: 704,600 nodes and 20,536,900 relationships ingested in **50.4 seconds** (407,000 relationships/second).

3. Constraints & Indexes:

- Uniqueness constraints on `Customer.customer_id`, `Account.account_id`, `Device.device_id`.
- Range index on `TRANSFER.occurred_at`: Allows rapid temporal filtering in Cypher without full edge scanning.

3. Routine Business Queries (Q1 to Q3) Line-by-Line

3.1 Q1: Account Activity (Balance vs. Outgoing/Incoming Volume)

- **Objective:** For a given suspect account, calculate total outgoing volume, total incoming volume, transfer counts, and active period.
- **SQL Implementation (`queries.py`):**

```
  SELECT
      COALESCE(SUM(CASE WHEN sender_id = %(acc)s THEN amount ELSE 0 END), 0) AS sent_amount,
      COALESCE(SUM(CASE WHEN receiver_id = %(acc)s THEN amount ELSE 0 END), 0) AS
received_amount,
      COUNT(CASE WHEN sender_id = %(acc)s THEN 1 END) AS sent_count,
      COUNT(CASE WHEN receiver_id = %(acc)s THEN 1 END) AS received_count
  FROM transfers
  WHERE sender_id = %(acc)s OR receiver_id = %(acc)s;
```

- **Line Analysis:**
- `WHERE sender_id = %(acc)s OR receiver_id = %(acc)s`: PostgreSQL combines the two B-Tree indexes (`idx_transfers_sender` and `idx_transfers_receiver`) using a `BitmapOr`.
- `CASE WHEN`: Separates incoming vs. outgoing amounts in a single pass without querying the table twice.
- **Cypher Implementation (`queries.py`):**

```
  MATCH (a:Account {account_id: $acc})
  OPTIONAL MATCH (a)-[out:TRANSFER]->()
  OPTIONAL MATCH ()-[in:TRANSFER]->(a)
```

```
RETURN
    coalesce(sum(out.amount), 0.0) AS sent_amount,
    coalesce(sum(in.amount), 0.0) AS received_amount,
    count(out) AS sent_count,
    count(in) AS received_count
```

- **Line Analysis:**
- `MATCH (a:Account {account_id: $acc})`: Uses unique index lookup ($O(1)$) to locate node `a`.
- Neo4j traverses only the relationship pointers directly connected to node `a`. It does not touch the remaining 20 million transfers.

3.2 Q2: Daily Activity and Month-Long Reconciliations

- **Objective:** Group 20,003,300 transfers into 28 daily buckets for September 2026, computing daily transfer count, gross volume, and fees.
- **SQL Implementation:**

```
SELECT
    occurred_at::date AS day,
    COUNT(*) AS total_transfers,
    SUM(amount) AS total_volume,
    SUM(ROUND(amount * 0.015, 2)) AS total_fees
FROM transfers
GROUP BY occurred_at::date
ORDER BY day;
```

- **Why PostgreSQL crushes Neo4j here (5.83 s vs. 38.63 s):**
- PostgreSQL spawns 2 parallel workers (`max_parallel_workers_per_gather = 2`).
- It performs a blazing-fast **Parallel Sequential Scan** reading sequential 8KB blocks off NVMe storage at >500 MB/s.
- Each worker aggregates its partition into a local hash table, and the Gather Merge node consolidates the 28 daily totals in milliseconds.
- Neo4j, by contrast, must scan 20,003,300 relationship records, deserialize their property stores in Java JVM heap, extract `occurred_at` and `amount`, and group them single-threadedly, incurring substantial object allocation and memory bandwidth overhead.

3.3 Q3: Shared Devices and Temporal Overlaps

- **Objective:** Given an account, find all other accounts that have used the same physical device where their active usage periods overlap.
- **SQL Implementation:**

```
SELECT
    ad2.account_id AS shared_account_id,
    ad1.device_id,
    GREATEST(ad1.first_seen, ad2.first_seen) AS overlap_start,
    LEAST(ad1.last_seen, ad2.last_seen) AS overlap_end
FROM account_devices ad1
JOIN account_devices ad2 ON ad1.device_id = ad2.device_id AND ad1.account_id != ad2.account_id
```

```
  WHERE ad1.account_id = %(acc)s
    AND ad1.first_seen <= ad2.last_seen
    AND ad1.last_seen >= ad2.first_seen;
```

- **Line Analysis:**
- `JOIN account_devices ad2 ON ad1.device_id = ad2.device_id`: Self-join on device.
- `ad1.first_seen <= ad2.last_seen AND ad1.last_seen >= ad2.first_seen`: Standard mathematical interval overlap condition. Two intervals $[S_1, E_1]$ and $[S_2, E_2]$ overlap if and only if $S_1 \le E_2$ and $E_1 \ge S_2$.
- **Cypher Implementation:**

```
  MATCH (a1:Account {account_id: $acc})-[u1:USED_DEVICE]->(d:Device)<-[u2:USED_DEVICE]-
(a2:Account)
  WHERE a1 <> a2
    AND u1.first_seen <= u2.last_seen
    AND u1.last_seen >= u2.first_seen
  RETURN a2.account_id, d.device_id,
         CASE WHEN u1.first_seen > u2.first_seen THEN u1.first_seen ELSE u2.first_seen END AS
overlap_start,
         CASE WHEN u1.last_seen < u2.last_seen THEN u1.last_seen ELSE u2.last_seen END AS
overlap_end
```

- **Line Analysis:**
- Declarative path: `(a1)-[:USED_DEVICE]->(d)<-[:USED_DEVICE]-(a2)` traverses the device bipartite structure naturally without explicit table self-joins.

4. Forensic Network Workloads (Q4 to Q7) Line-by-Line

4.1 Q4: Ownership and Device Networks

- **Objective:** Multi-hop network expansion tracing accounts reachable through alternating ownership and device sharing edges up to depth $k \in \{4, 6, 8, 10\}$.
- **SQL Implementation:** Uses a Recursive Common Table Expression (CTE) tracking an array of visited accounts `ARRAY[start_acc]` to prevent infinite cycling.
- **Cypher Implementation:**

```
  MATCH (a:Account {account_id: $acc})
  MATCH path = (a)-[:OWNS|USED_DEVICE*..$k]-(other:Account)
  WHERE other <> a
  RETURN DISTINCT other.account_id AS reached_account
```

- **Performance Trade-Off:**
- As depth increases from $k=4$ to $k=10$, small-world network expansion causes the number of visited nodes to grow exponentially. At $k=10$, nearly 240,000 accounts are connected due to weak ties (e.g. shared agent devices). Neo4j traverses paths using relationship pointers, outperforming SQL recursive self-joins by 20x.

4.2 Q5: Rapid Onward Payments (Layering Chains)

- **Objective:** Find 3-hop transfer chains $A \to B \to C \to D$ occurring within 300 seconds (5 minutes), where each transfer retains at least 80% of the preceding amount ($v_{next} \ge 0.8 v_{prev}$).
- **SQL Implementation:**

```
  SELECT
      t1.transfer_id AS t1_id, t1.amount AS a1, t1.occurred_at AS time1,
      t2.transfer_id AS t2_id, t2.amount AS a2, t2.occurred_at AS time2,
      t3.transfer_id AS t3_id, t3.amount AS a3, t3.occurred_at AS time3
  FROM transfers t1
  JOIN transfers t2 ON t1.receiver_id = t2.sender_id
  JOIN transfers t3 ON t2.receiver_id = t3.sender_id
  WHERE t1.sender_id = %(acc)s
    AND t2.occurred_at >= t1.occurred_at AND t2.occurred_at <= t1.occurred_at + INTERVAL '300
seconds'
    AND t3.occurred_at >= t2.occurred_at AND t3.occurred_at <= t1.occurred_at + INTERVAL '300
seconds'
    AND t2.amount >= t1.amount * 0.80 AND t2.amount <= t1.amount
    AND t3.amount >= t2.amount * 0.80 AND t3.amount <= t2.amount;
```

- **Cypher Implementation:**

```
  MATCH (a:Account {account_id: $acc})-[t1:TRANSFER]->(b:Account)-[t2:TRANSFER]->(c:Account)-
[t3:TRANSFER]->(d:Account)
  WHERE t2.occurred_at >= t1.occurred_at
    AND duration.inSeconds(t1.occurred_at, t3.occurred_at).seconds <= 300
    AND t2.amount >= t1.amount * 0.80 AND t2.amount <= t1.amount
    AND t3.amount >= t2.amount * 0.80 AND t3.amount <= t2.amount
  RETURN t1.transfer_id, t2.transfer_id, t3.transfer_id, t3.amount;
```

4.3 Q6: Circular Payments (Round-Tripping / Wash Transfers)

- **Objective:** Detect closed transfer loops originating at account A and returning to A within 3,600 seconds (1 hour), returning at least 70% of the initial capital ($v_{return} \ge 0.70 v_{start}$).
- **SQL Implementation:** Uses recursive CTE with `path_nodes` array checking `receiver_id = start_acc`.
- **Cypher Implementation:**

```
  MATCH (a:Account {account_id: $acc})
  MATCH path = (a)-[:TRANSFER*2..6]->(a)
  WITH path, relationships(path) AS txs
  WHERE duration.inSeconds(txs[0].occurred_at, txs[-1].occurred_at).seconds <= 3600
    AND txs[-1].amount >= txs[0].amount * 0.70
  RETURN [node IN nodes(path) | node.account_id] AS cycle_accounts,
         [tx IN txs | tx.amount] AS cycle_amounts,
         duration.inSeconds(txs[0].occurred_at, txs[-1].occurred_at).seconds AS elapsed_seconds;
```

4.4 Q7: Shortest Path and the `q7_v2` Algorithmic Optimization

- **The Problem:** In Q7 bound-6, we seek the shortest directed transfer path between suspect and control accounts.
- **SQL v1 (Recursive CTE):**

- Explores paths recursively: `WITH RECURSIVE search_graph AS (...)`.
- **The Flaw:** In SQL v1, if an account has already been reached at depth 2, the recursive CTE continues to expand it again at depths 3, 4, 5, and 6 along other higher-cost paths.
- As a result, PostgreSQL reads **14,600,000 transfer rows** and produces 427,955 intermediate rows for a network of only 240,000 accounts! Latency explodes to **33,668.3 ms**.
- **The Optimization (`sql/05_q7_v2.sql`):**

We replaced the single recursive CTE with an iterative level-by-level Breadth-First Search (BFS) procedure utilizing a temporary visited table:

```
CREATE OR REPLACE FUNCTION find_shortest_path_v2(p_src INT, p_dst INT, p_max_depth INT)
RETURNS TABLE(hop_count INT, path INT[]) AS $$
DECLARE
    v_depth INT := 0;
    v_found BOOLEAN := FALSE;
BEGIN
    CREATE TEMP TABLE IF NOT EXISTS visited_acc (
        account_id INT PRIMARY KEY,
        depth INT,
        path INT[]
    ) ON COMMIT DROP;
    TRUNCATE visited_acc;

    INSERT INTO visited_acc VALUES (p_src, 0, ARRAY[p_src]);

    WHILE v_depth < p_max_depth AND NOT v_found LOOP
        v_depth := v_depth + 1;

        -- Expand exactly one level forward
        INSERT INTO visited_acc (account_id, depth, path)
        SELECT DISTINCT t.receiver_id, v_depth, v.path || t.receiver_id
        FROM visited_acc v
        JOIN transfers t ON v.account_id = t.sender_id
        WHERE v.depth = v_depth - 1
        ON CONFLICT (account_id) DO NOTHING;  -- Crucial: prune already-visited accounts!

        -- Stop immediately once destination is reached
        IF EXISTS (SELECT 1 FROM visited_acc WHERE account_id = p_dst) THEN
            v_found := TRUE;
            EXIT;
        END IF;
    END LOOP;

    RETURN QUERY
    SELECT v.depth, v.path FROM visited_acc v WHERE v.account_id = p_dst;
END;
$$ LANGUAGE plpgsql;
```

- **Why `q7_v2` Wins at Bound 6:**
- `ON CONFLICT (account_id) DO NOTHING`: Ensures each account is expanded **at most once**, completely eliminating redundant path exploration.
- `EXIT`: Stops search immediately when destination is reached at level 5, rather than blindly continuing to depth 6.
- Result: PostgreSQL bound-6 execution drops from **33,668 ms to 5,045 ms** (a 6.7x speedup).

- **Why `q7_v2` Loses at Bound 4 (345 ms vs. 468 ms):**
- For shallow searches ($k=4$), the fixed overhead of creating, truncating, and indexing the temporary table `visited_acc` in `plpgsql` exceeds the small amount of work pruned.

5. Concurrency Load Testing (`code/concurrency.py`) Line-by-Line

5.1 Architecture of the Concurrency Test

`concurrency.py` simulates production workloads where multiple forensic analysts execute queries simultaneously:

- **Clients:** 1, 4, 8, 16 parallel processes spawned via Python's `multiprocessing.Process`.
- **Target Workload:** Q7 shortest-path queries executed continuously in a loop for a 15-second measurement window.
- **Metrics Recorded:** Completed cycles, errors, timeouts (>10 s), throughput (cycles/sec), median latency, and 95th percentile (p95) latency.

5.2 Saturation Analysis: Why PostgreSQL Flattens and Neo4j Soars

- **Hardware Profile:** Intel Core i5-8250U has **4 physical cores and 8 logical threads** (`NumberOfLogicalProcessors = 8`).
- **PostgreSQL Saturation:**
- PostgreSQL uses a process-per-connection model.
- At 1 client: 0.309 cycles/s (latency 3.2 s).
- At 4 clients: 0.650 cycles/s.
- At 8 clients: 0.911 cycles/s (all 8 hardware threads 100% utilized).
- At 16 clients: **0.911 cycles/s** (throughput completely flatlines).
- **Latency Impact:** Because CPU resources are exhausted, 16 clients queue behind each other, causing median latency to double from 12.1 s to 23.3 s, and p95 latency to jump to 29.4 s.
- **Neo4j Scaling:**
- Because each Q7 traversal requires only 18.7 ms of CPU time, Neo4j handles 8 clients with ease, scaling to **836.6 cycles/s** (over 900x higher throughput than PostgreSQL).

6. Forensic Investigation & Regulatory Decision Logic (Section 8)

6.1 Suspect Ranking Matrix

1. **Rank 1: Suspect Account 206978 (Set C):**

- **Evidence:** Closed 6-transfer circular payment cycle:

$206978 \to 206979 \to 206980 \to 206981 \to 206982 \to 206983 \to 206978$.

- **Velocity:** Completes full round-trip in **200 seconds** (average 40 seconds per hop).
- **Value Decay:** Starts with 300,000 RWF; each hop subtracts exactly 2.0% (standard mobile money agent fee schedule):

$300,000 \to 294,000 \to 288,120 \to 282,358 \to 276,710 \to 271,176$ RWF. Total returned: 90.4% of original capital.

- **Device Co-Location Smoking Gun:** All five intermediary accounts (`206979` through `206983`) were accessed from the **identical physical handset (Device `228043`)** during the exact timestamp of the cycle!
- **Forensic Assessment:** Classic automated layering and smurfing ring operated by a single entity using multiple SIM cards on one phone.

2. Rank 2: Suspect Account 998 (Set A):

- 4-transfer cycle circulating 200,000 RWF over 120 seconds on handset `228001`.

3. Rank 3: Suspect Account 147228 (Set B):

- Rapid onward layering (1.05M RWF), but no return loop. Likely rapid payroll or supplier cash-out.

6.2 Ordinary Business Explanations & Due Process

Before issuing freezing orders under Rwandan anti-money laundering (AML/CFT) laws, investigators must eliminate legitimate commercial explanations:

- **Agent Float Rebalancing:** Cash-in/cash-out agents frequently transfer electronic float among themselves or use a merchant aggregator when physical cash runs low.
- **Cooperative Liquidity Pools:** Agricultural cooperatives (e.g. tea/coffee farmers in Western Province) circulate operational funds across member accounts.
- **Required Missing Evidence:**

1. Know Your Customer (KYC) identity documentation (checking if account holders share national ID numbers or family ties).

2. Cellular Base Transceiver Station (BTS) tower triangulation (confirming if phones were physically in the same room).

3. Customer dispute and fraud complaints filed with telco operators.

7. Study Guide Summary for Group Members

Member	Primary Focus Areas	Key Defense Talking Points
Amie Marie Flora Dushimumukiza (101405)	Ingestion pipeline, Relational schemas, Bulk loading, Data audit	Explain why \copy and dropping indexes is essential for 20M rows; explain the SHA-256 generator verification.
Kwizera Rene (101379)	Graph modeling, q7_v2 BFS SQL rewrite, Concurrency testing	Explain why Neo4j Index-Free Adjacency wins at Q7; explain why q7_v2 uses visited temp tables to prune 14M rows; explain 8-thread CPU saturation.
Grace Teta (101378)	Business queries Q1–Q3, Forensic analysis, Regulatory decisions	Explain why PostgreSQL beats Neo4j on Q2 daily aggregation (parallel sequential scans); explain the 6-hop cycle and device co-location on Account 206978.

End of Line-by-Line Code & Project Technical Explanation Guide